5.3 EzloPi Dynamic Config via BLE

- 1 Config JSON Overview
- 2 BLE Protocol Details
  - 2.1 Write Config Data
  - 2.2 Read Config Data

This documentation explains the BLE protocol for writing and reading of the config JSON.

1. Config JSON Overview

Config JSON is something that contains the GPO and peripheral configurations. The JSON below is an example of config JSON and the array **dev_detail** contains the JSON item for each of the device being configured for EzloP

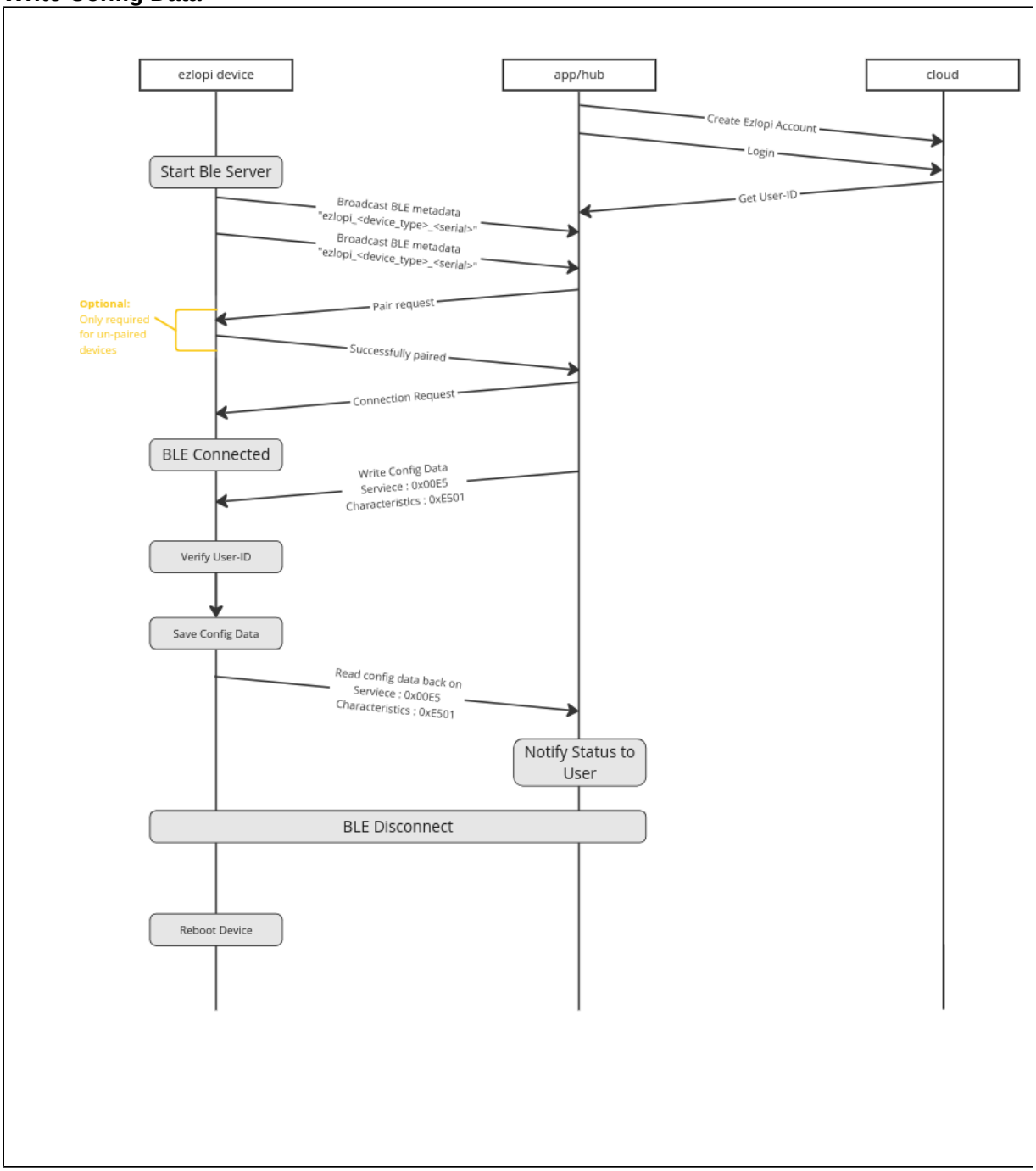
Config JSON

```
{
    "config_id": "_ID_",
    "config_time": 1696508363,
    "config_name": "My moisture sensor",
    "chipset": "ESP32", // ESP32, ESP32S3 or ESP32C3
    "dev_detail": [
        {
            "dev_type": 1,
            "dev_name": "Dining Room Main Lamp",
            "id_room": "",
            "id_item": 2,
            "val_ip": true,
            "val_op": false,
            "gpio_in": 21,
            "gpio_out": 13,
            "is_ip": false,
            "ip_inv": true,
            "pullup_ip": true,
            "pullup_op ": true,
            "op_inv": false
        },
        {
            "dev_type": 2,
            "dev_name": "Switch",
            "id_room": "",
            "id_item": 4,
            "gpio": 21,
            "pull_up": true,
            "logic_inv": true
        },
        {
            "dev_type": 8,
            "dev_name": "Sensor for my Dog",
            "id_room": "",
            "id_item": 5,
            "gpio_sda": 21,
            "gpio_scl": 22,
            "pullup_scl": true,
            "pullup_sda": true,
            "slave_addr": 105
        }
    ]
}
```

2. BLE Protocol Details

The use can configure for GPIOs and peripherals for EzloPi using the UI, and we generate the config JSON internally. The config JSON in can be sent to the EzloPi device via BLE based on a service and a characteristics. Service for config data read write is 0x00E5 and characteristics is 0xE501. The BLE flow for connection and read/write of config is shown in the figure below.

a. **Write Config Data**



Before sharing the config data via BLE we minify and encode it into bsae64 characters.
The minified JSON can be as :

Minified Config Data

```
{"config_id":"_ID_","config_time":1696508363,"chipset": "ESP32","config_name":"My moisture
sensor","dev_detail":[{"dev_type":1,"dev_name":"Dining Room Main Lamp","id_room":"","id_item":2,"
val_ip":true,"val_op":false,"gpio_in":21,"gpio_out":13,"is_ip":false,"ip_inv":true,"pullup_ip":
true,"pullup_op ":true,"op_inv":false},{"dev_type":2,"dev_name":"Switch","id_room":"","id_item":4,"
gpio":21,"pull_up":true,"logic_inv":true},{"dev_type":8,"dev_name":"Sensor for my Dog","
id_room":"","id_item":5,"gpio_sda":21,"gpio_scl":22,"pullup_scl":true,"pullup_sda":true,"
slave_addr":105}]}
```

The minified JSON is then converted to Base64 encoded string.

Base64 Encoded Config Data

```
eyJjb25maWdfaWQiOiJfSURfIiwiY29uZmlnX3RpbWUiOjE2OTY1MDgzNjMsImRldl90b3RhbCI6MywiY29uZmlnX25hbWUiOiJ
NeSBtb2lzdHVyZSBzZW5zb3IiLCJkZXZfZGV0YWlsIjpbeyJkZXZfdHlwZSI6MSwiZGV2X25hbWUiOiJEaW5pbmcgUm9vbSBNYW
luIExhbXAiLCJpZF9yb29tIjoiIiwiaWRfaXRlbSI6MiwidmFsX2lwIjp0cnVlLCJ2YWxfb3AiOmZhbHNlLCJncGlvX2luIjoyM
SwiZ3Bpb19vdXQiOjEzLCJpc19pcCI6ZmFsc2UsImlwX2ludiI6dHJ1ZSwicHVsbHVwX2lwIjp0cnVlLCJwdWxsdXBfb3AgIjp0
cnVlLCJvcF9pbnYiOmZhbHNlfSx7ImRldl90eXBlIjoyLCJkZXZfbmFtZSI6IlN3aXRjaCIsImlkX3Jvb20iOiIiLCJpZF9pdGV
tIjo0LCJncGlvIjoyMSwicHVsbF91cCI6dHJ1ZSwibG9naWNfaW52Ijp0cnVlfSx7ImRldl90eXBlIjo4LCJkZXZfbmFtZSI6Il
NlbnNvciBmb3IgbXkgRG9nIiwiaWRfcm9vbSI6IiIsImlkX2l0ZW0iOjUsImdwaW9fc2RhIjoyMSwiZ3Bpb19zY2wiOjIyLCJwd
WxsdXBfc2NsIjp0cnVlLCJwdWxsdXBfc2RhIjp0cnVlLCJzbGF2ZV9hZGRyIjoxMDV9XX0=
```

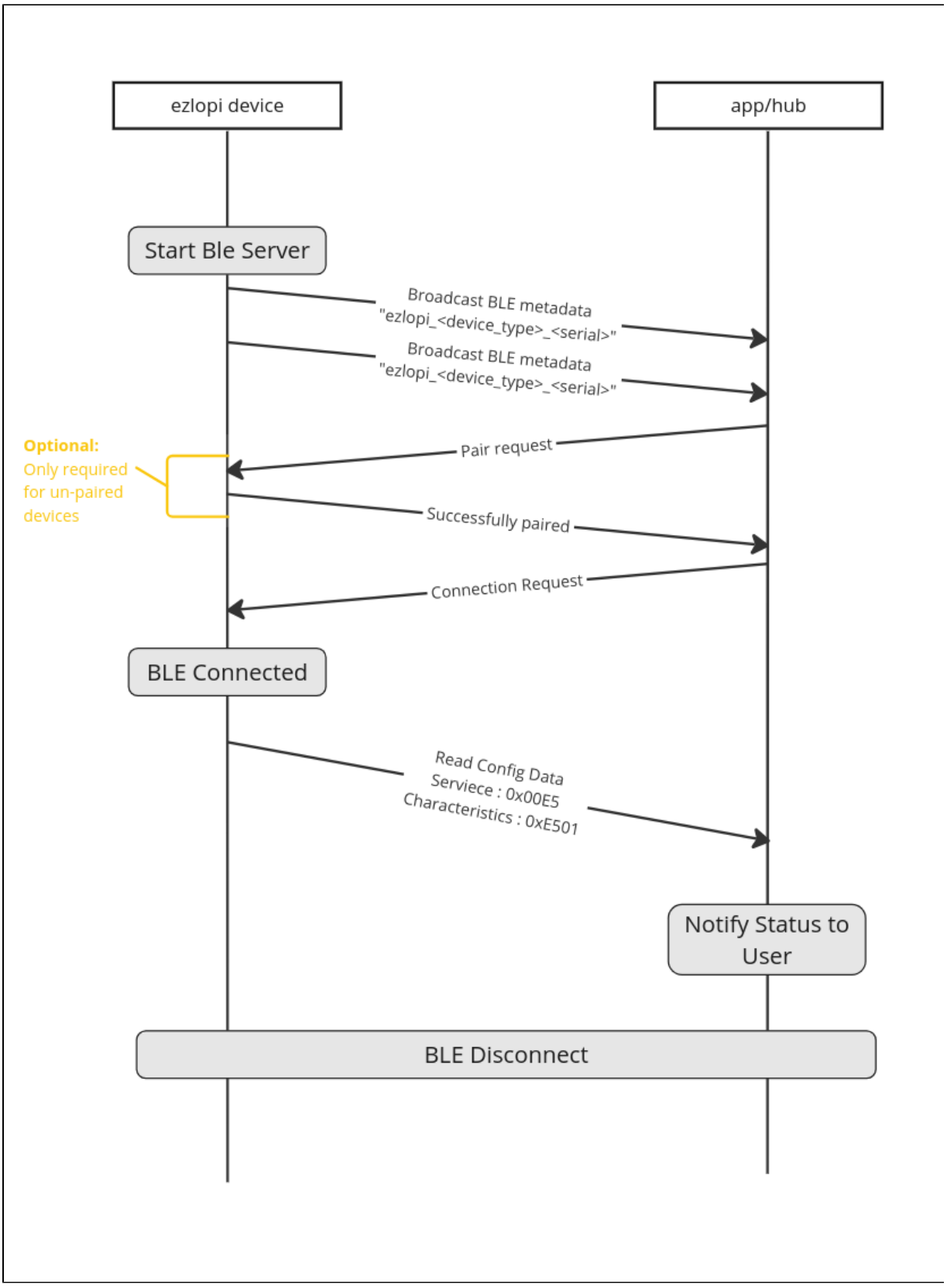
This base64 characters then sent with 400 characters each time as :

Single Chunk

```
{
  "total_len": 753, // Total length of base64 string
  "len": 400,        // Length of the chunk
  "sequence": 1,     // data sequence number
  "data":
"eyJzZXJpYWwiOjEwMDAwNDk2MSwicHJvdmlzaW9uX3NlcnZlciI6Imh0dHBzOi8vcmVxLWRpc3AtYXQwbS5taW9zLmNvbS8iLC
JjbG91ZF9zZXJ2ZXIiOiJodHRwczovL2Nsb3VkLmV6bG8uY29tOjcwMDAiLCJwcm92aXNpb25fdG9rZW4iOiJjMjU0MTVhNWYxZ
GU5ZTUwODRkYWI1YjBjOTJhMjhkMmY1ZjY0YmIwMDc5YTlmZmNjMzEyZWIwMDVmZjFkMmI5YmI4OTZhMDk3YTU2NWViNzlmYjE1
MzM0ZjdjMjk2ZThjYTY3NTk2Y2RlOGE4NzI1NWQ0NGUyNTA3ZmFjZTNjMjg4YWVkOTI2MjRlMzU1MjA3NzAwYjA3MDcxMzY1MzU
zYWNj"
}
```

b. ## Read Config Data

Config Data can be read using the same Service and characteristics as we write. Service for config data read write is 0x00E5 and characteristics is 0xE501. Mobile or Hub can verify the data comparing the **config_id** or **date**.



The minified JSON can be as :

Minified Config Data


```
"config_id":"_ID_","config_time":1696508363,"chipset": "ESP32","config_name":"My moisture sensor","
dev_detail":[{"dev_type":1,"dev_name":"Dining Room Main Lamp","id_room":"","id_item":2,"val_ip":
true,"val_op":false,"gpio_in":21,"gpio_out":13,"is_ip":false,"ip_inv":true,"pullup_ip":true,"
pullup_op ":true,"op_inv":false},{"dev_type":2,"dev_name":"Switch","id_room":"","id_item":4,"gpio":
21,"pull_up":true,"logic_inv":true},{"dev_type":8,"dev_name":"Sensor for my Dog","id_room":"","
id_item":5,"gpio_sda":21,"gpio_scl":22,"pullup_scl":true,"pullup_sda":true,"slave_addr":105}]}
```


The minified JSON is then converted to Base64 encoded string.

Base64 Encoded Config Data


```
eyJjb25maWdfaWQiOiJfSURfIiwiY29uZmlnX3RpbWUiOjE2OTY1MDgzNjMsImRldl90b3RhbCI6MywiZGV2X2RldGFpbCI6W3s
iZGV2X3R5cGUiOjEsImRldl9uYW1lIjoiRGluaW5nIFJvb20gTWFpbiBMYW1wIiwiaWRfcm9vbSI6IiIsImlkX2l0ZW0iOjIsIn
ZhbF9pcCI6dHJ1ZSwidmFsX29wIjpmYWxzZSwiZ3Bpb19pbiI6MjEsImdwaW9fb3V0IjoxMywiaXNfaXAiOmZhbHNlLCJpcF9pb
nYiOnRydWUsInB1bGx1cF9pcCI6dHJ1ZSwicHVsbHVwX29wICI6dHJ1ZSwib3BfaW52IjpmYWxzZX0seyJkZXZfdHlwZSI6Miwi
ZGV2X25hbWUiOiJTd2l0Y2giLCJpZF9yb29tIjoiIiwiaWRfaXRlbSI6NCwiZ3BpbyI6MjEsInB1bGxfdXAiOnRydWUsImxvZ2l
jX2ludiI6dHJ1ZX0seyJkZXZfdHlwZSI6OCwiZGV2X25hbWUiOiJTZW5zb3IgZm9yIG15IERvZyIsImlkX3Jvb20iOiIiLCJpZF
9pdGVtIjo1LCJncGlvX3NkYSI6MjEsImdwaW9fc2NsIjoyMiwicHVsbHVwX3NjbCI6dHJ1ZSwicHVsbHVwX3NkYSI6dHJ1ZSwic
2xhdmVfYWRkciI6MTA1fV19
```


This base64 characters then sent with 400 characters each time as :

Single Chunk


```
{
  "total_len": 753, // Total length of base64 string
  "len": 400,        // Length of the chunk
  "sequence": 1,     // data sequence number
  "data":
"eyJzZXJpYWwiOjEwMDAwNDk2MSwicHJvdmlzaW9uX3NlcnZlciI6Imh0dHBzOi8vcmVxLWRpc3AtYXQwbS5taW9zLmNvbS8iLC
JjbG91ZF9zZXJ2ZXIiOiJodHRwczovL2Nsb3VkLmV6bG8uY29tOjcwMDAiLCJwcm92aXNpb25fdG9rZW4iOiJjMjU0MTVhNWYxZ
GU5ZTUwODRkYWI1YjBjOTJhMjhkMmY1ZjY0YmIwMDc5YTlmZmNjMzEyZWIwMDVmZjFkMmI5YmI4OTZhMDk3YTU2NWViNzlmYjE1
MzM0ZjdjMjk2ZThjYTY3NTk2Y2RlOGE4NzI1NWQ0NGUyNTA3ZmFjZTNjMjg4YWVkOTI2MjRlMzU1MjA3NzAwYjA3MDcxMzY1MzU
zYWNj"
}
```